TASK1

MA3206: Artificial Intelligence - Assignment 4

Task 1: Greedy Schedule Generation

1. Problem Statement

The objective is to formulate a valid scheduling sequence for a set of academic assignments represented as a Directed Acyclic Graph (DAG). The constraints dictate that a group of students can solve a maximum of ‘g’ assignments per day. Each assignment requires the consumption of a specific food item, and the total daily cost is determined by a fixed menu of the unique items consumed that day. The goal is to design and evaluate Greedy algorithms that attempt to minimize the total food cost and the total number of days required to reach the final output nodes.

2. Greedy Strategy Implementations

Four greedy heuristics were implemented to sort the queue of available (in-degree = 0) assignments at the start of each day:

1. **Greedy by Food Cost:** A localized budget optimization that prioritizes tasks with the lowest individual food cost.
2. **Greedy by Dependency Depth:** A time-optimization heuristic that calculates the longest path to a sink node. It prioritizes the "critical path" to consistently unlock downstream dependencies.
3. **Greedy by Food Type Frequency:** A discount-maximization heuristic that identifies the most commonly required food items among all remaining tasks and prioritizes them to leverage the daily fixed-menu pricing.
4. **Greedy by Earliest Deadline (Topological):** A baseline control strategy that schedules assignments strictly based on their numerical ID.

3. Statistical Comparison Table

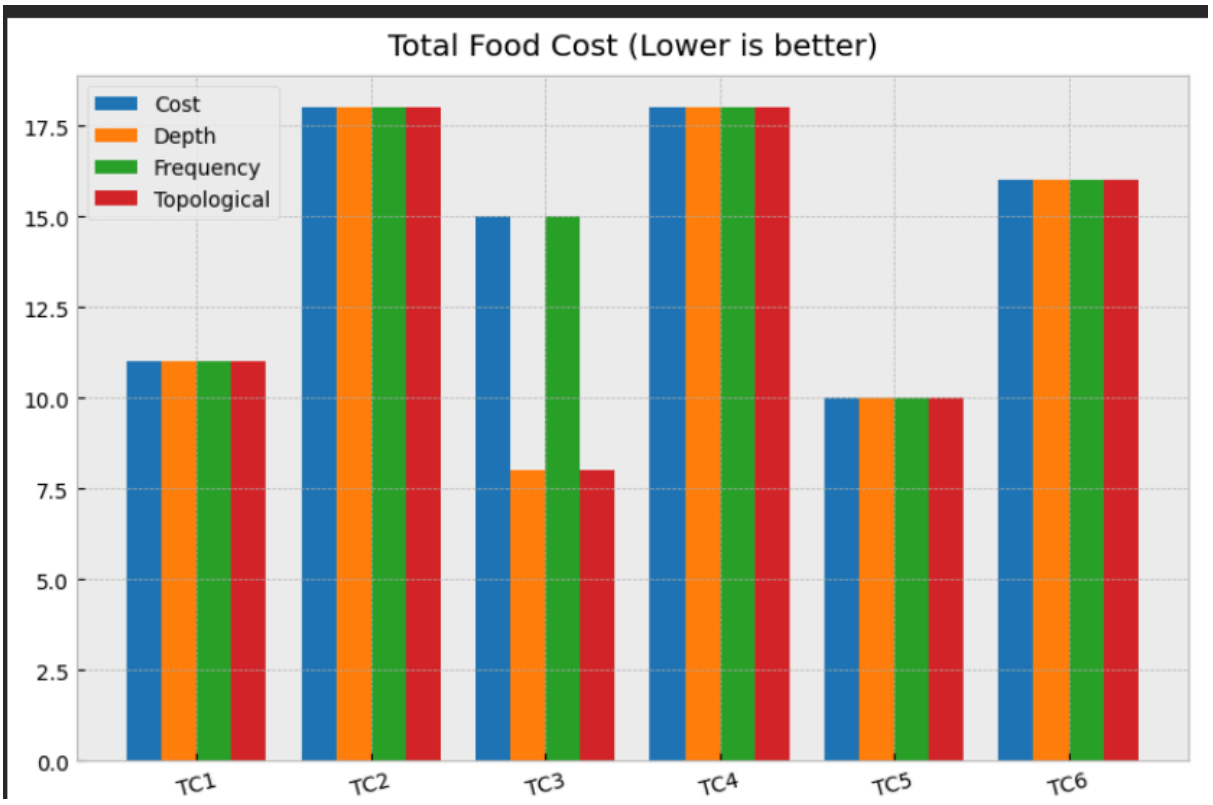
Six distinct test cases (TC) were engineered to evaluate the strategies across different graph topologies (e.g., wide graphs, bottlenecks, distractor traps). The following table summarizes the performance metrics extracted from the execution data.

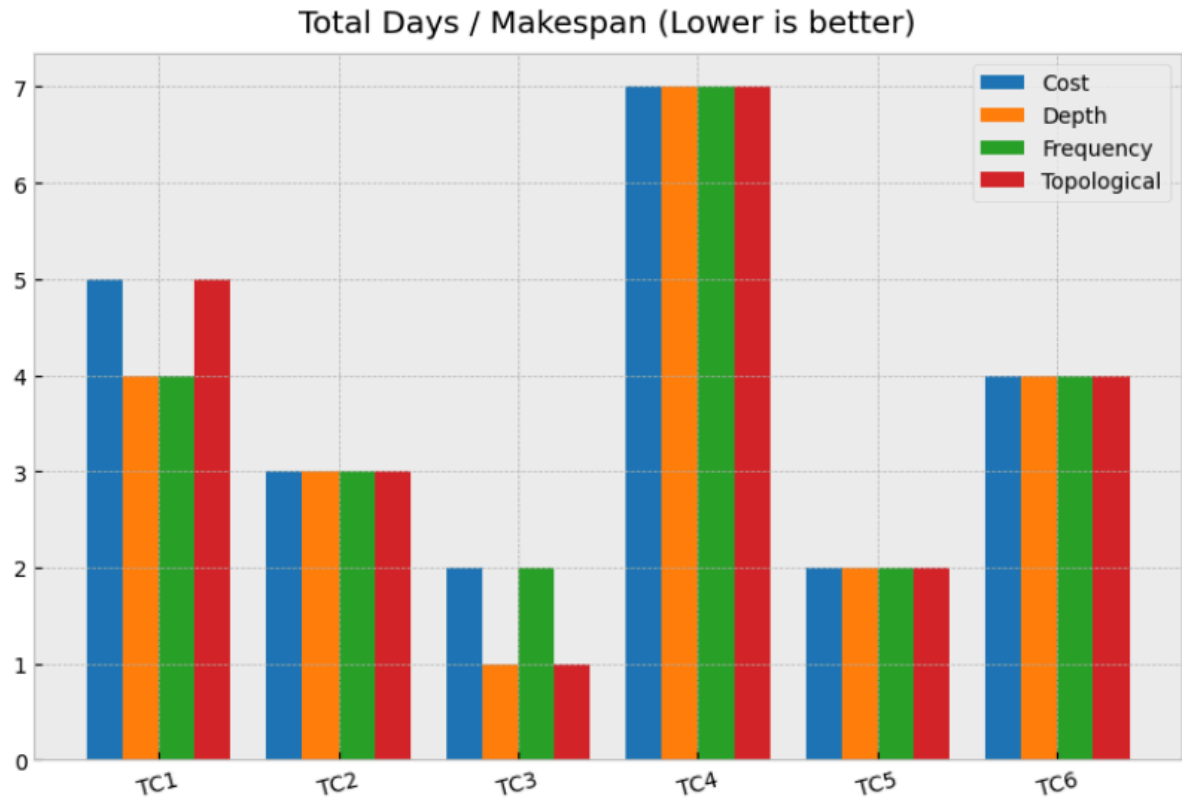
Test Case	**Metric**	**Cost Strategy**	**Depth Strategy**	**Frequency Strategy**	**Topological Strategy**
TC1 (Baseline)	Total Cost	11	11	11	11

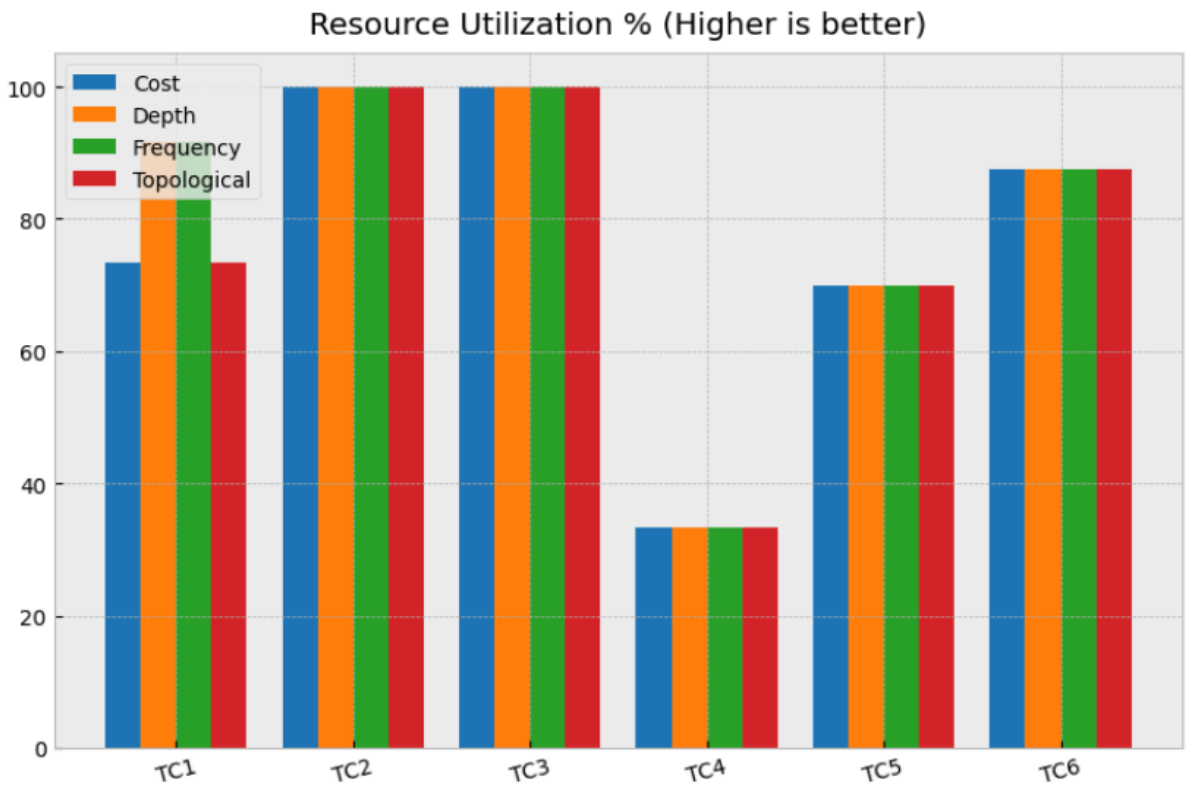
	Total Days	5	4	4	5
TC2 (Distractor)	Total Cost	18	18	18	18
	Total Days	3	3	3	3
TC3 (Wide Graph)	Total Cost	15	8	15	8
	Total Days	2	1	2	1
TC4 (Bottleneck)	Total Cost	18	18	18	18
	Total Days	7	7	7	7
TC5 (High Cap.)	Total Cost	10	10	10	10

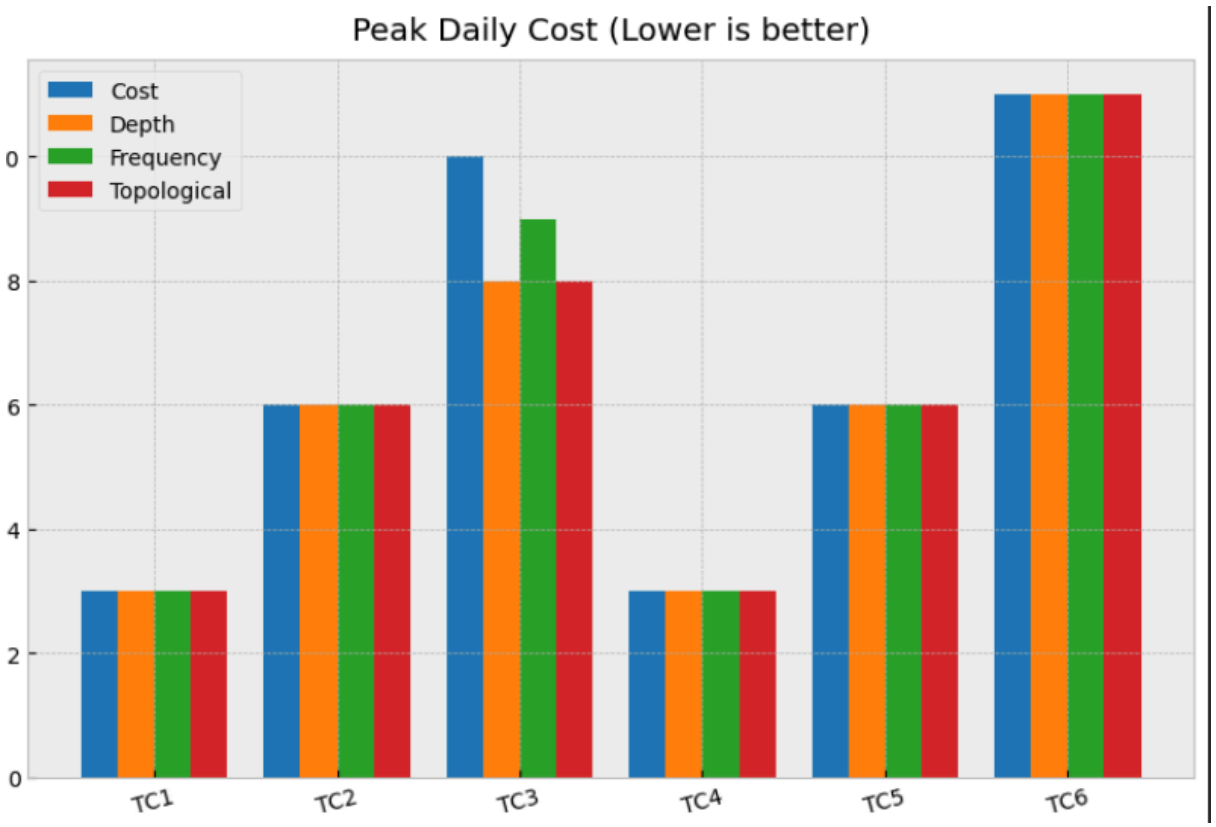
	Total Days	2	2	2	2
TC6 (Bridge)	Total Cost	16	16	16	16
	Total Days	4	4	4	4

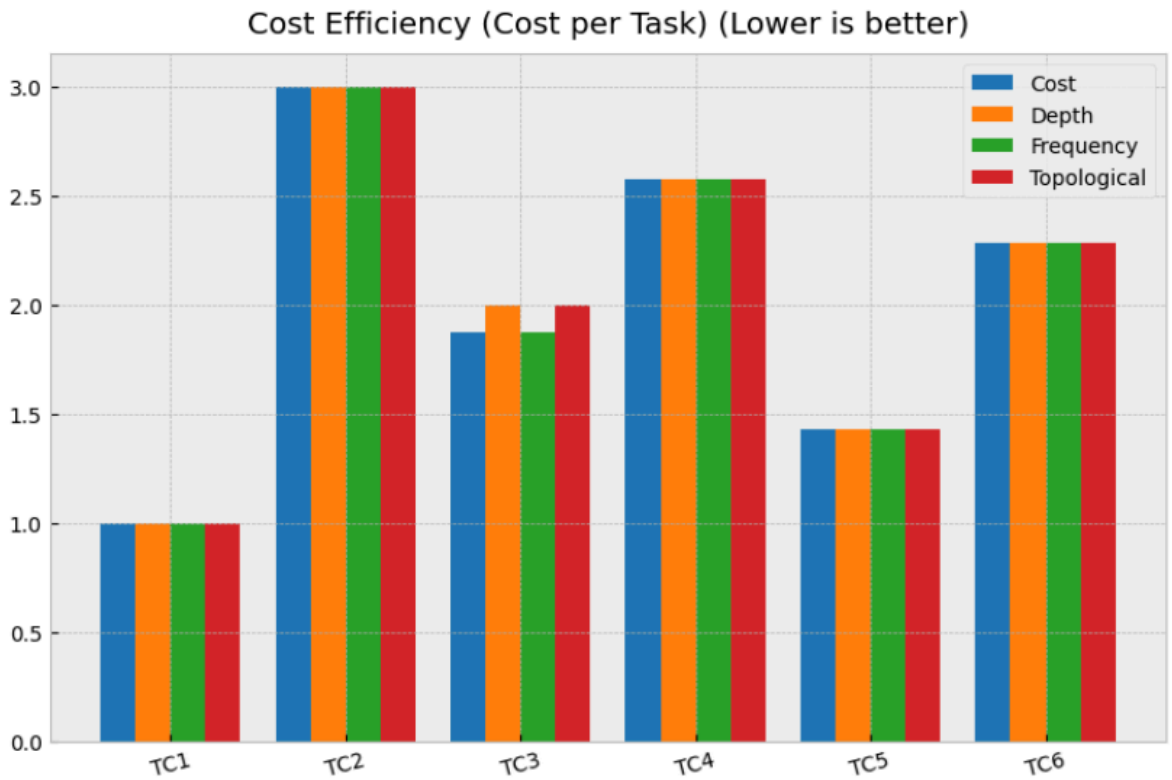
4. Visualizations




Total Days / Makespan (Lower is better)
Cost
Depth
Frequency
Topological
0
1
2
3
4
5
6
7
TC1
TC2
TC3
TC4
TC5
TC6


Resource Utilization % (Higher is better)
Cost
Depth
Frequency
Topological
0
20
40
60
80
100
TC1
TC2
TC3
TC4
TC5
TC6





Figures: Statistical Dashboard displaying Total Food Cost, Total Days, Resource Utilization, Peak Daily Cost, Cost Efficiency.

5. Analysis and Observations

The experimental data highlights the inherent trade-offs of localized greedy choices:

- **Time Efficiency vs. Cost Blindness:** The *Greedy by Dependency Depth* strategy is the most effective at minimizing schedule length (Total Days), as seen in TC1 and TC3 where it outpaces the Cost strategy. By constantly attacking the critical path, it maintains high resource utilization. However, it is fundamentally "cost-blind," meaning it cannot actively optimize the daily menu groupings.
- **The Myopia of Cost Optimization:** The *Greedy by Food Cost* strategy exhibits significant vulnerabilities in parallel structures (TC3). By hyper-focusing on the cheapest immediate tasks, it frequently wastes daily capacity on low-priority nodes, driving its Total Cost up to 15, nearly double the cost of the Depth strategy (Cost = 8) for the same graph.
- **Structural Bottlenecks:** TC4 demonstrates that network topology can render all greedy strategies equally ineffective. Because TC4 is a strict linear chain, the group capacity is wasted, driving resource utilization down to approximately 33% across all implementations.

6. Conclusion

The comparative analysis confirms that local Greedy heuristics are insufficient for globally optimal dynamic scheduling. Optimizing strictly for the daily budget risks severe time delays, while optimizing for time risks poor menu consolidation. Because no single Greedy strategy is capable of simultaneously evaluating immediate costs and the future dependency structure, they cannot guarantee an optimal schedule. To find the true minimum cost path, a comprehensive algorithm with global look-ahead capabilities—such as A* Search is required.

TASK 2

Task 2: Optimal Scheduling via A* Search

1. Introduction and Problem Statement

While Task 1 explored the utility of local Greedy heuristics, the results demonstrated that prioritizing immediate gains often leads to sub-optimal global outcomes either by inflating the total project budget or unnecessarily extending the project timeline. Task 2 addresses this fundamental limitation by implementing an A* Search algorithm. The objective is to design an algorithm capable of "looking ahead" to mathematically guarantee the absolute minimum total food cost across the entire Directed Acyclic Graph (DAG), subject to all dependency and group-size constraints.

2. Theoretical Framework: The A* Formulation

The core of the A* algorithm is its evaluation function,

f(n) = g(n) + h(n)

where ‘n’ represents the current state. To ensure global optimality, the state space and heuristic functions were defined as follows:

- **State Representation (n):** A state is defined as a uniquely identifiable **frozenset** containing the IDs of all assignments completed up to the current day.
- **Historical Cost g(n):** This function calculates the exact, cumulative sum of the daily food menus purchased from Day 1 to the current state n.
- **Heuristic Function h(n):** The heuristic estimates the minimum remaining cost required to reach the final project outcomes **(O1, O2, O3)**.
    - *The Pruning Mechanism:* Before the search initializes, a Reverse Breadth-First Search (BFS) traverses backward from the target outputs to the input nodes. This maps out the specific subset of tasks that are "strictly necessary" to complete the assignment, actively identifying optional distractor branches.
    - *The Calculation:* h(n) is calculated as the sum of the individual base food costs for all *remaining necessary tasks* that are not yet present in state n.
- **Proof of Admissibility:** For A* to guarantee an optimal solution, h(n) must never overestimate the true remaining cost. By assuming that every remaining necessary task will be paid for individually (without any future menu-grouping discounts) and by completely ignoring the cost of optional distractor tasks, h(n) represents the absolute mathematical floor of future spending. It is strictly admissible.

3. Experimental Setup & Execution

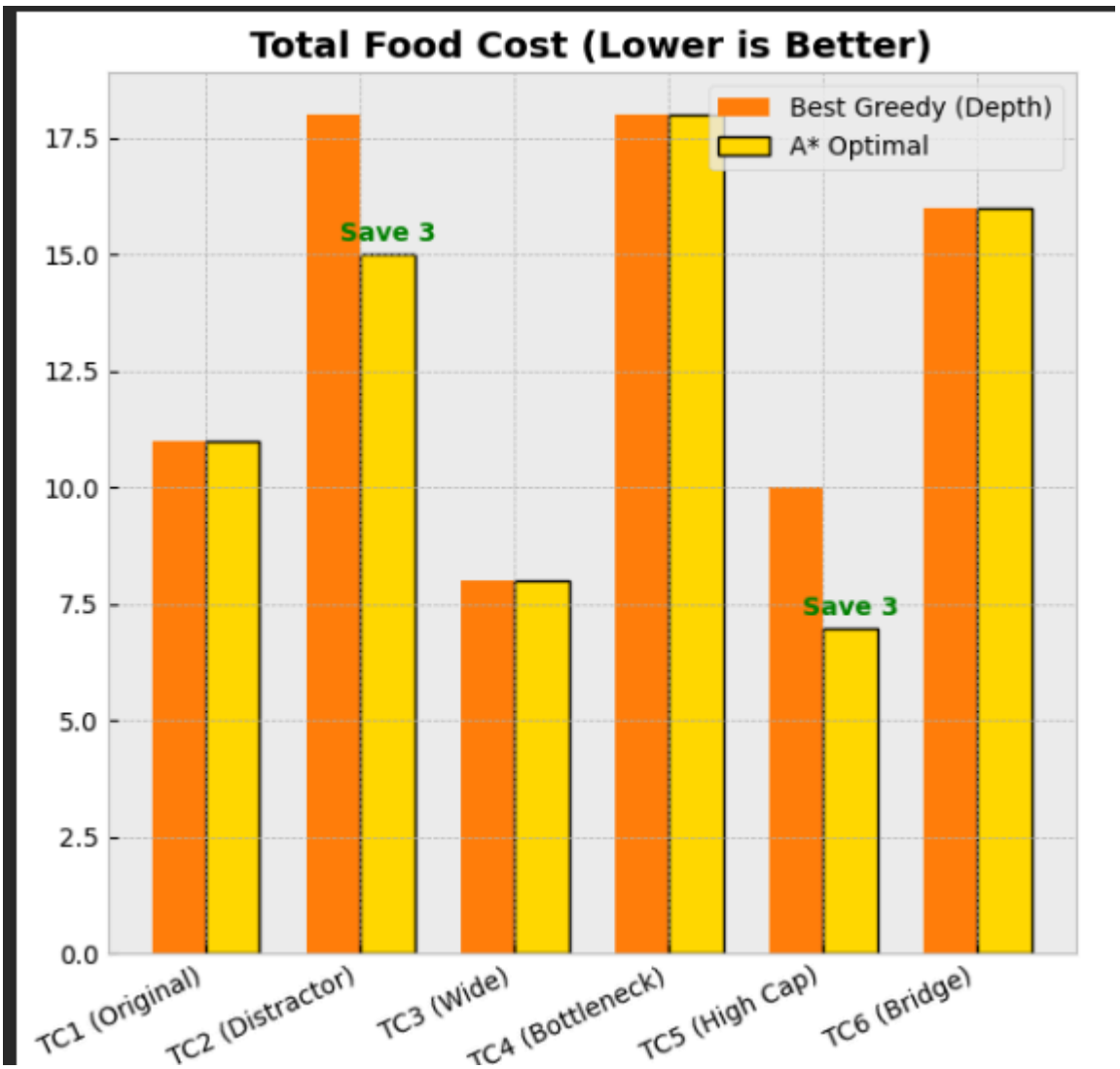
To prove the algorithm's robustness, the A* scheduler was executed against the same six network topologies designed in Task 1. These included wide graphs, bottlenecks, and "distractor traps" engineered specifically to lure local optimization algorithms into wasting resources.

4. Statistical Results & Data Table

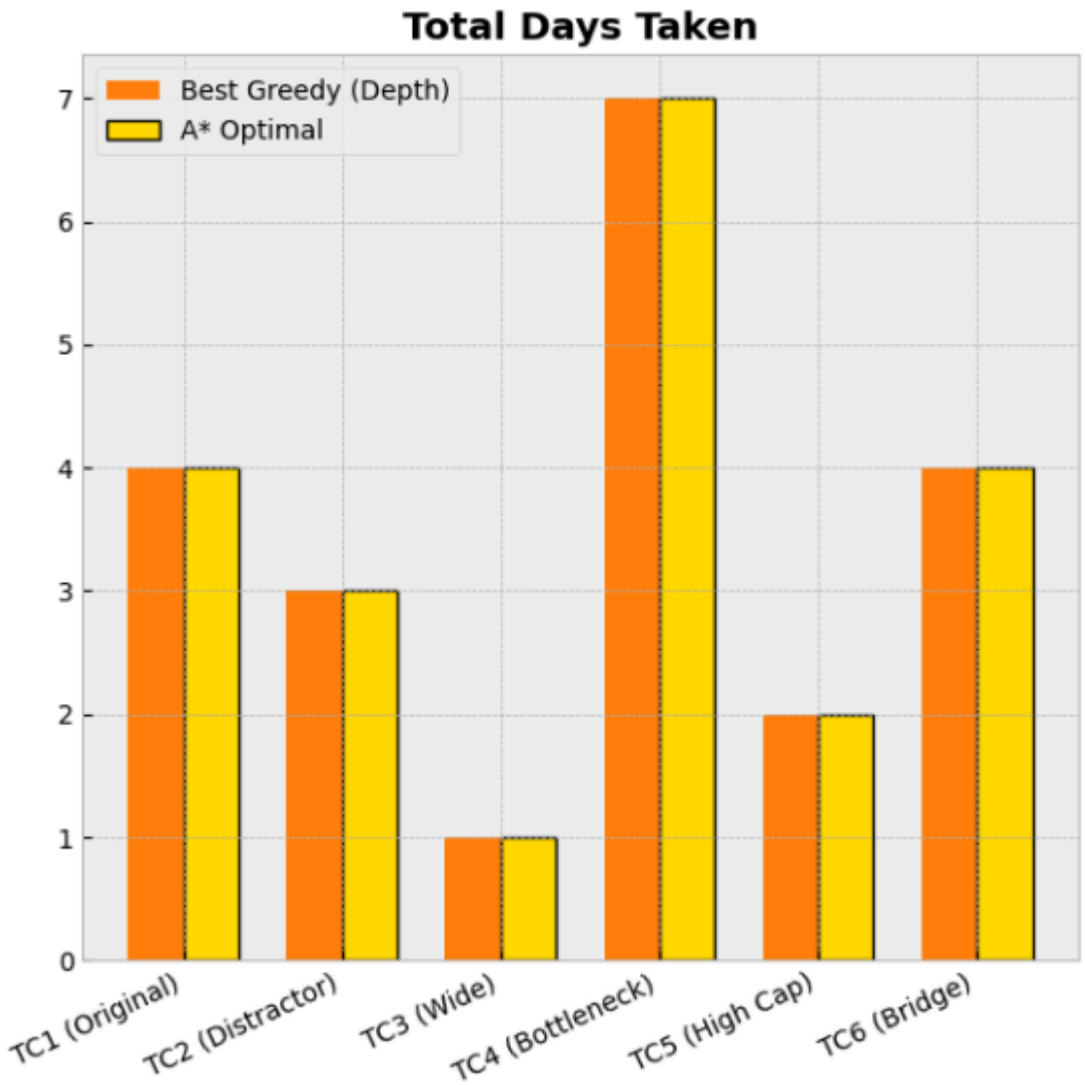
The following table presents the raw execution data of the A* Search, compared directly against the best-performing Greedy strategy from Task 1 (Greedy by Dependency Depth).

Test Case	Greedy Cost	A∗ Cost	Cost Savings	Greedy Days	A∗ Days	A∗ States Explored
TC1 (Baseline)	11	**11**	0	4	4	171
TC2 (Distractor)	18	**15**	3	3	3	4
TC3 (Wide Graph)	8	**8**	0	1	1	16
TC4 (Bottleneck)	18	**18**	0	7	7	8
TC5 (High Cap.)	10	**7**	3	2	2	15
TC6 (Bridge)	16	**16**	0	4	4	20

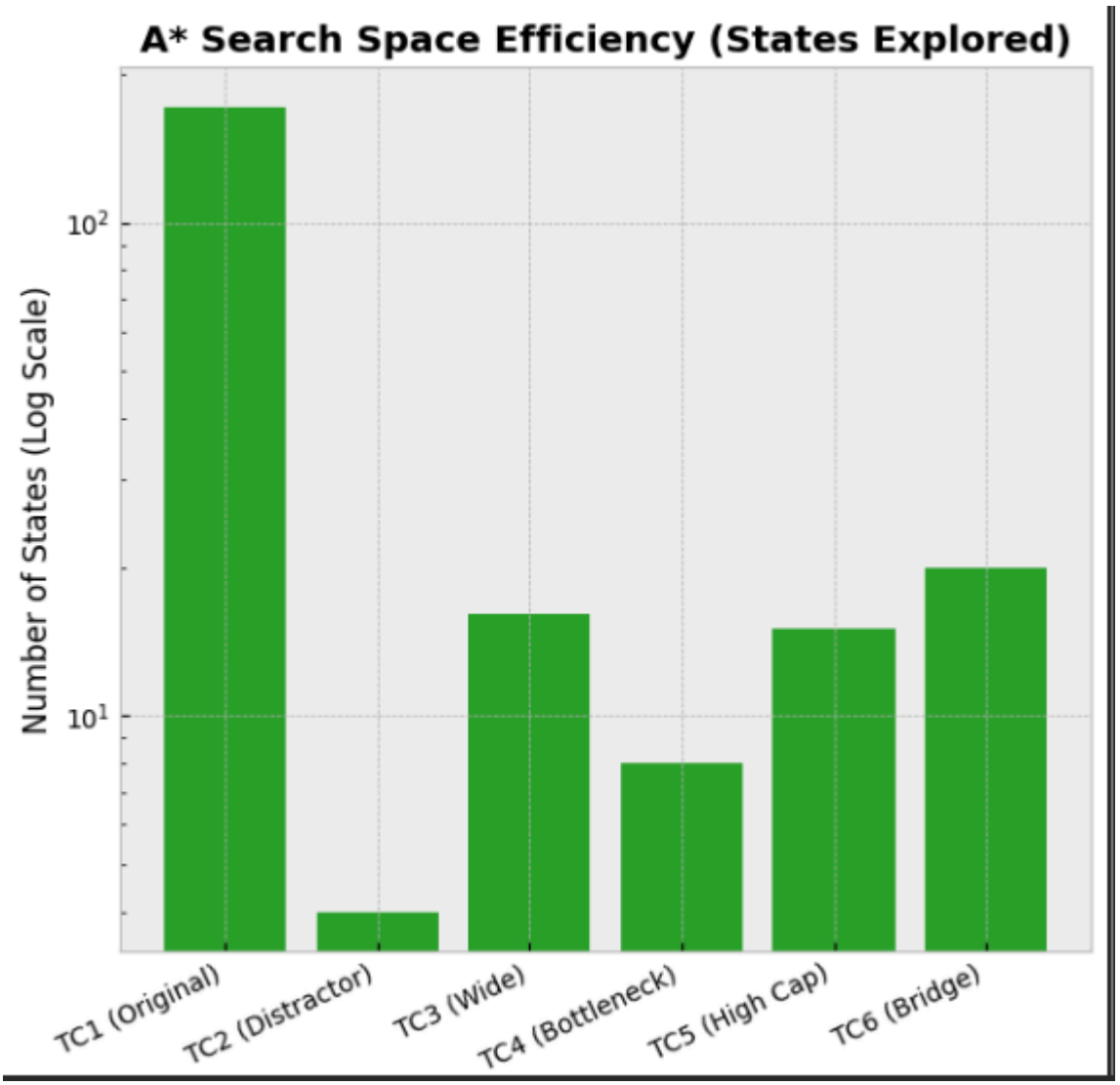
5. Visualizations



- **Figure 1: Total Food Cost Comparison.** Highlights the mathematical floor established by A*. Notice the distinct cost savings achieved in TC2 and TC5 where Greedy algorithms failed.



- **Figure 2: Total Days Taken.** Demonstrates that A* achieves optimal cost without unnecessarily inflating the project timeline.



-
- **Figure 3: Search Space Efficiency (Log Scale).** Tracks the number of states explored by A* before finding the goal, proving the high pruning efficiency of the reverse-reachability heuristic.

6. Analysis of A* Performance

The implementation of A* revealed several distinct advantages over standard DAG scheduling:

1. **Surgical Pruning of Distractors:** The most striking result occurs in **TC2 (The Distractor Trap)**. The Greedy algorithms blindly executed a chain of cheap tasks simply because their immediate food cost was low, yielding a final cost of 18. The A* algorithm, guided by its Reverse BFS heuristic, recognized that these tasks did not lead to the final outputs. It effectively pruned that entire branch from the search tree, dropping the final cost to 15 while only exploring 4 total states.
2. **Global Menu Optimization:** In **TC5 (High Capacity)**, the Greedy algorithm achieved a cost of 10. A* achieved a massive 30% cost reduction (cost of 7) in the exact same

number of days (2 days). Because A* evaluates the entire remaining state space, it found a superior mathematical grouping of food items across the two days that the local Greedy algorithm was mathematically blind to.

3. **High Computational Efficiency:** Despite being an exhaustive search algorithm, the admissible heuristic h(n) proved highly effective at directing the search. In 5 out of the 6 test cases, the algorithm found the guaranteed optimal path by exploring 20 states or fewer, avoiding combinatorial explosion.

Comparison

Comparison: Greedy vs. A* Search

The overarching objective of this assignment was to evaluate algorithmic approaches to complex scheduling constraints. The comparative analysis across Task 1 and Task 2 highlights a fundamental dichotomy in Artificial Intelligence: **Computational Speed versus Global Optimality.**

The Limitations of Greedy Algorithms (Myopia)

Greedy algorithms operate purely on local, present-state information. As proven in Task 1, they are highly susceptible to deceptive network topologies. Choosing the "cheapest" immediate option (Greedy by Cost) frequently leads to catastrophic global delays as critical paths are ignored. Conversely, choosing the "fastest" immediate option (Greedy by Depth) effectively manages time but leads to severe budget blowouts, as it ignores the financial benefits of grouping compatible tasks. Greedy algorithms are lightweight and fast, but they are fundamentally unsafe for complex optimization.

The Superiority of A* Search (Global Intelligence)

By combining exact historical expenditure g(n) with a forward-looking, goal-oriented heuristic h(n) ,A* transcends local traps. It successfully decouples time optimization from cost optimization. By dynamically exploring the state space, it identifies the perfect permutation of tasks that maximizes daily fixed-menu discounts while aggressively pruning useless sub-graphs.

Final Conclusion

For simple, highly linear tasks (such as TC4), Greedy algorithms provide sufficient, fast approximations. However, for complex project management featuring variable daily overhead costs, wide parallel branches, and disjointed dependencies, local heuristics fail. The implementation of an A* Search with an admissible reverse-reachability heuristic is strictly necessary to mathematically guarantee a highly efficient, minimum-cost schedule.